

Common Problems

WhatsApp

 Real-time UpdatesBy Stefan Mai · Updated Feb 16, 2026 ·  medium · Asked at: 

Watch Video Walkthrough

Watch the author walk through the problem step-by-step

 Watch Now

Understanding the Problem

What is Whatsapp?

Whatsapp is a messaging service that allows users to send and receive encrypted messages and calls from their phones and computers. Whatsapp is famously built on Erlang and renowned for handling high scale with limited engineering and infrastructure outlay.

Functional Requirements

Apps like WhatsApp and Messenger have tons of features, but your interviewer doesn't want you to cover them all. The most obvious capabilities are almost definitely in-scope but it's good to ask your interviewer if they want you to move beyond. Spending too much time in requirements will make it harder for you to give detail in the rest of the interview, so we won't dawdle too long here!

Core Requirements

1. Users should be able to start group chats with multiple participants (limit 100).
2. Users should be able to send/receive messages.
3. Users should be able to receive messages sent while they are not online (up to 30 days).
4. Users should be able to send/receive media in their messages.



That third requirement isn't obvious to everyone (but it's interesting to design) and If I'm your interviewer I'll probably guide you to it.

Below the line (out of scope)

1. Audio/Video calling.
2. Interactions with businesses.
3. Registration and profile management.

Non-Functional Requirements

Before getting into non-functional requirements, it might make sense to ask your interviewer how the app is used by the majority of users if you haven't used it much. Are users mostly doing 1:1 chats, or is the app for large groups? How often are people sending messages? These questions will help you to understand the system that needs to be built and while they are not explicitly "requirements" they will dictate some design decisions that come later.

Core Requirements

1. Messages should be delivered to available users with low latency, < 500ms.
2. We should guarantee deliverability of messages - they should make their way to users.
3. The system should be able to handle billions of users with high throughput (we'll estimate later).
4. Messages should be stored on centralized servers no longer than necessary.
5. The system should be resilient against failures of individual components.

Below the line (out of scope)

1. Exhaustive treatment of security concerns.
2. Spam and scraping prevention systems.



Adding features that are out of scope is a "nice to have". It shows product thinking and gives your interviewer a chance to help you reprioritize based on what they want to see in the interview. That said, it's very much a nice to have. If additional features are not coming to you quickly (or you've already burned some time), don't waste your time and move on.

It's easy to use precious time defining features that are out of scope, which provides negligible value for a hiring decision.

Functional

- Start group chats (limit 100 users)
- Send-receive messages
- Send/receive media
- Access messages after they've been offline

Non-Functional

- Delivered with low-latency, < 500ms
- Guarantee delivery of messages
- Billions of users, high throughput
- Messages not stored unnecessarily
- Fault-tolerant

Requirements

The Set Up

Planning the Approach

Before you move on to designing the system, it's important to start by taking a moment to plan your strategy for the session. For this problem, we might first recognize that 1:1 messages are simply a special case of larger chats (with 2 participants), so we'll solve for that general case of group messages even while we focus on the 1:1 case. We can also reflect a little and acknowledge that part of the design will be able durably delivering messages to users, and another is about doing so in realtime.

After this, we should be able to start our design by walking through our core requirements and solving them as simply as possible. This will get us started with a system that is probably slow and not scalable, but a good starting point for us to optimize in the deep dives.

In our deep dives we'll address scaling, optimizations, and any additional features/functionality the interviewer might want to throw on the fire.

Defining the Core Entities

In the core entities section, we'll think through the main "nouns" of our system. The intent here is to give us the right language to reason through the problem and set the stage for our API and data model.



Interviewers aren't evaluating you on what you list for core entities, they're an intermediate step to help you reason through the problem. That doesn't mean they don't matter though! Getting the entities wrong is a great way to start building on a broken foundation - so spend a few moments to get them right and keep moving.

We can walk through our functional requirements to get an idea of what the core entities are. We need:

- Users
- Chats (2-100 users)
- Messages
- Clients (a user might have multiple devices)

We'll use this language to reason through the problem.

API or System Interface

Next, we'll want to think through the API of our system. Unlike a lot of other products where a REST API is probably appropriate, for a chat app, we're going to have high-frequency updates being both sent and received. This is a perfect use case for a bi-directional socket connection!

⚡ Pattern: Real-time Updates

WebSocket connections and real-time messaging demonstrate the broader **real-time updates pattern** used across many distributed systems. Whether it's chat messages, live dashboards, collaborative editing, or gaming, the same principles apply: persistent connections for low latency, pub/sub for scaling across servers, and careful state management for reliability.

[Learn This Pattern](#)

For this interview, we'll use WebSockets (over TLS for security), although a custom protocol over a raw TLS-encrypted TCP connection would also work. The idea will be that users will open

the app and connect to the server, opening this socket which will be used to send and receive commands which represent our API.

As we define our API, we'll specify the commands that are sent and received over the connection by the client.

First, let's be able to create a chat.

```
// -> createChat
{
  "participants": [],
  "name": ""
} -> {
  "chatId": ""
}
```



Now we should be able to send messages on the chat.

```
// -> sendMessage
{
  "chatId": "",
  "message": "",
  "attachments": []
} -> {
  "status": "SUCCESS" | "FAILURE",
  "messageId": ""
}
```



We need a way to create attachments (note: I'm going to amend this later in the writeup).

```
// -> createAttachment
{
  "body": ...,
  "hash":
} -> {
  "attachmentId": ""
}
```



And we need a way to add/remove users to the chat.

```
// -> modifyChatParticipants
{
  "chatId": "",
  "userId": "",
  "operation": "ADD" | "REMOVE"
} -> "SUCCESS" | "FAILURE"
```



Each of these commands will have parallel commands that are sent to other clients. When the command has been received by clients, they'll send an `ack` command back to the server letting it know the command has been received (and it doesn't have to be sent again)!



The message receipt acknowledgement is a bit non-obvious but crucial to making sure we don't lose messages. By forcing clients to ack, we can know for certain that the message has been delivered all the way to the client.

When a chat is created or updated ...

```
// <- chatUpdate
{
  "chatId": "",
  "participants": [],
} -> "RECEIVED"
```



When a message is received ...

```
// <- newMessage
{
  "chatId": "",
  "userId": "",
  "message": "",
  "attachments": []
} -> "RECEIVED"
```



Etc ...

Note that enumerating all of these APIs can take time! In the actual interview, I might shortcut by only writing the command names and not the full API. It's also usually a good idea to summarize the API initially before you build out the high-level design in case things need to change. "I'll come back to this as I learn more" is completely acceptable!

Our whiteboard might look like this:

Commands Sent

- * createChat
- * sendMessage
- * createAttachment
- * modifyParticipants

Commands Received

- * newMessage
- * chatUpdate

Commands Exchanged

Now that we have a base to work with let's figure out how we can implement them while we satisfy our requirements.

High-Level Design

1) Users should be able to start group chats with multiple participants (limit 100)

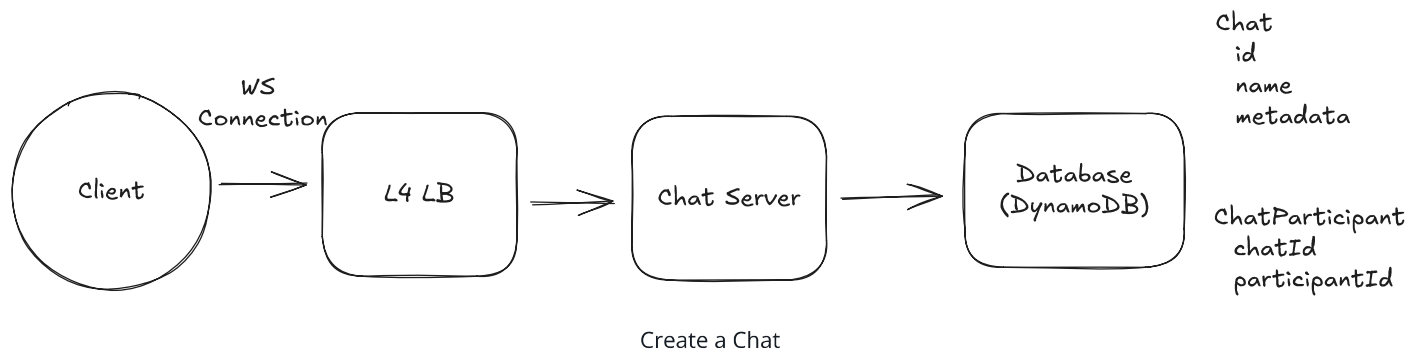
For our first requirement, we need a way for a user to create a chat. We'll start with a simple service behind an L4 load balancer (we're using Websockets) which can write `chat` metadata to a database. Let's use DynamoDB for fast key/value performance and scalability here, although we have lots of other options.



Can we use an L7 load balancer? In many cases, yes. There is wide support for Websockets in many modern L7 load balancers. But the important thing is that *we don't need any L7 capabilities for this service*. L7 load balancers shine when we want to, for instance, route traffic with specific paths or headers to different services. They are also helpful when we

may want to spread HTTP requests across many servers even behind a single client connection. But neither of these apply here!

So using an L4 load balancer is sufficient and will generally be higher performance than a L7 load balancer.



The steps here are:

1. User connects to the service and sends a `createChat` message.
2. The service creates a `chat` record in the database along with a `ChatParticipant` record for each user in the chat. For small chats this can be done in a single DynamoDB transaction (up to 100 items), but for chats near the 100-participant limit we may need to batch the writes.
3. The service returns the `chatId` to the user.

On the chat table, we'll usually just want to look up the details by the chat's ID. Having a simple primary key on the chat `id` is good enough for this.

For the `ChatParticipant` table, we'll want to be able to (1) look up all participants for a given chat and (2) look up all chats for a given user.

1. We can do this with a composite primary key where `chatId` is the partition key and `participantId` is the sort key. A Query on the `chatId` partition key will return all participants for a given chat.
2. We'll need a **Global Secondary Index (GSI)** with `participantId` as the partition key and `chatId` as the sort key. This will allow us to efficiently query all chats for a given user. The GSI will automatically be kept in sync with the base table by DynamoDB.

Great! We got some chats. How about messages?

2) Users should be able to send/receive messages.

To allow users to send/receive messages, we're going to need to start taking advantage of the websocket connection that we established. To keep things simple while we get off the ground, let's assume we have a single host for our Chat Server.

This is obviously a terrible solution for scale (and you might say so to your interviewer to keep them from itching), but it's a good starting point that will allow us to incrementally solve those problems as we go.



For infrastructure-style interviews, I highly recommend reasoning about a solution on a single node first. Oftentimes the path to scale is straightforward from there.

If you solve scale first without thinking about how the actual mechanics of your solution work underneath, you're more likely to back yourself into a corner.

When users make Websocket connections to our Chat Server, we'll want to keep track of their connection with a simple in-memory hash map which will map a user id to a websocket connection. This way we know which users are connected and can send them messages.

To send a message:

1. User sends a `sendMessage` message to the Chat Server.
2. The Chat Server looks up all participants in the chat via the `ChatParticipant` table.
3. The Chat Server looks up the websocket connection for each participant in its internal hash table and sends the message via each connection.

We're making some really strong assumptions here! We're assuming all users are online, connected to the same Chat Server, and that we have a websocket connection for each of them. But under those conditions this works! So let's keep going.

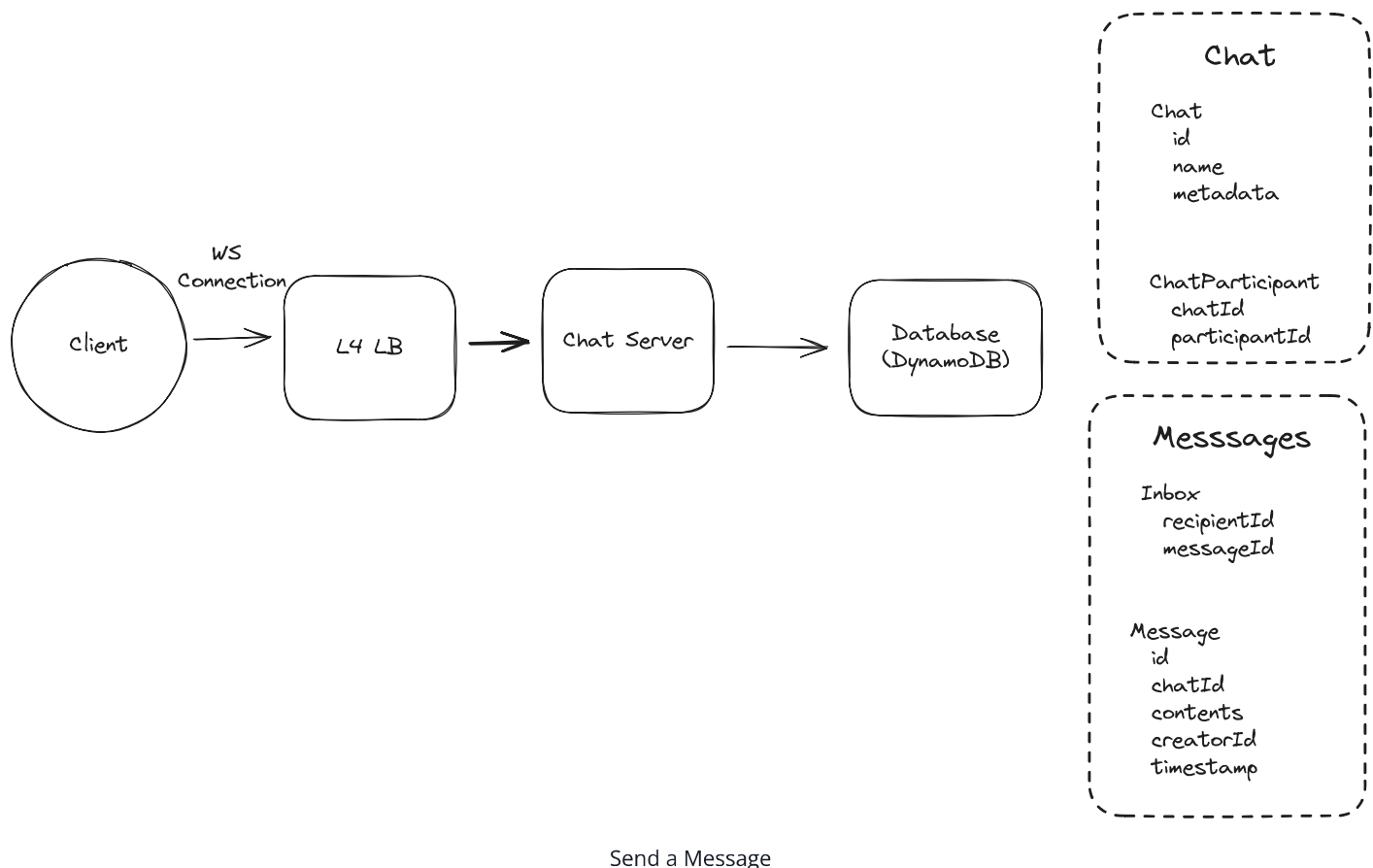
3) Users should be able to receive messages sent while they are not online (up to 30 days).

With our next requirement, we're going to need to start storing messages in our database so that we can deliver them to users even when they're offline. We'll take this as an opportunity to

add some robustness to our system.

Let's keep an "Inbox" for each user which will contain all undelivered messages. When messages are sent, we'll write them to the inbox of each recipient user. If they're already online, we can go ahead and try to deliver the message immediately. If they're not online, we'll store the message and wait for them to come back later.

How much write throughput does this add? The vast majority of chats are 1:1, and the average user sends about 20 messages per day. With 200M active users, that's 4B messages/day or roughly **40K messages/second**. For each message in a 1:1 chat, we write once to `Messages` and once to `Inbox` (for the recipient). Even accounting for group chats, we're looking at roughly **100K writes/second** — well within DynamoDB's capabilities with `userId` as the partition key.



So, to send a message:

1. Sender sends a `sendMessage` message to the Chat Server.
2. The Chat Server looks up all participants in the chat via the `ChatParticipant` table.
3. The Chat Server (a) writes the message to our `Message` table and (b) creates an entry in our `Inbox` table for each recipient.

4. The Chat Server returns a `SUCCESS` or `FAILURE` to the sender with the final message id.
5. The Chat Server looks up the websocket connection for each participant and attempts to deliver the message to each of them via `newMessage`.
6. (For connected clients) Upon receipt, the client will send an `ack` message to the Chat Server to indicate they've received the message. The Chat Server will then delete the message from the `Inbox` table.

For clients who aren't connected, we'll keep their messages in the `Inbox` table for some time. Later, when the client decides to connect, we'll:

1. Look up the user's `Inbox` and find any undelivered message IDs.
2. For each message ID, look up the message in the `Message` table.
3. Write those messages to the client's connection via the `newMessage` message.
4. Upon receipt, the client will send an `ack` message to the Chat Server to indicate they've received the message.
5. The Chat Server will then delete the message from the `Inbox` table.

Finally, we'll need to periodically clean up the old messages in the `Inbox` and messages tables. We can do this by setting a `TTL` on the items of the tables.

Great! We knocked out some of the durability issues of our initial solution and enabled offline delivery. Our solution still doesn't scale and we've got a lot more work to do, so let's keep moving.

4) Users should be able to send/receive media in their messages.

Our final requirement is that users should be able to send/receive media in their messages.

Users sending and receiving media is annoying. It's bandwidth- and storage- intensive. While we could potentially do this with our Chat Server and database, it's better to use purpose-built technologies for this. This is in fact how Whatsapp actually works: attachments are uploaded via a separate HTTP service.

Bad Solution: Keep attachments in DB



Good Solution: Send attachments via chat server



Great Solution: Manage attachments separately



Ok awesome, so we have a system which has real-time delivery of messages, persistence to handle offline use-cases, and attachments. It just doesn't scale ... yet!

Potential Deep Dives

With the core functional requirements met, it's time to dig into the non-functional requirements via deep dives and solve some of the issues we've earmarked to this point. This includes solving obvious scalability issues as well as auxiliary questions which demonstrate your command of system design.



The degree to which a candidate should proactively lead the deep dives is a function of their seniority. In this problem, all levels should be quick to point out that my single-host solution isn't going to scale. But beyond these bottlenecks, it's reasonable in a mid-level interview for the interviewer to drive the majority of the deep dives. However, in senior and staff+ interviews, the level of agency and ownership expected of the candidate increases. They should be able to proactively look around corners and identify potential issues with their design, proposing solutions to address them.

1) How can we handle billions of simultaneous users?

Our single-host system is convenient but unrealistic. Serving billions of users via a single machine isn't possible and it would make deployments and failures a nightmare. So what can we do? The obvious answer is to try to scale out the number of Chat Servers we have.

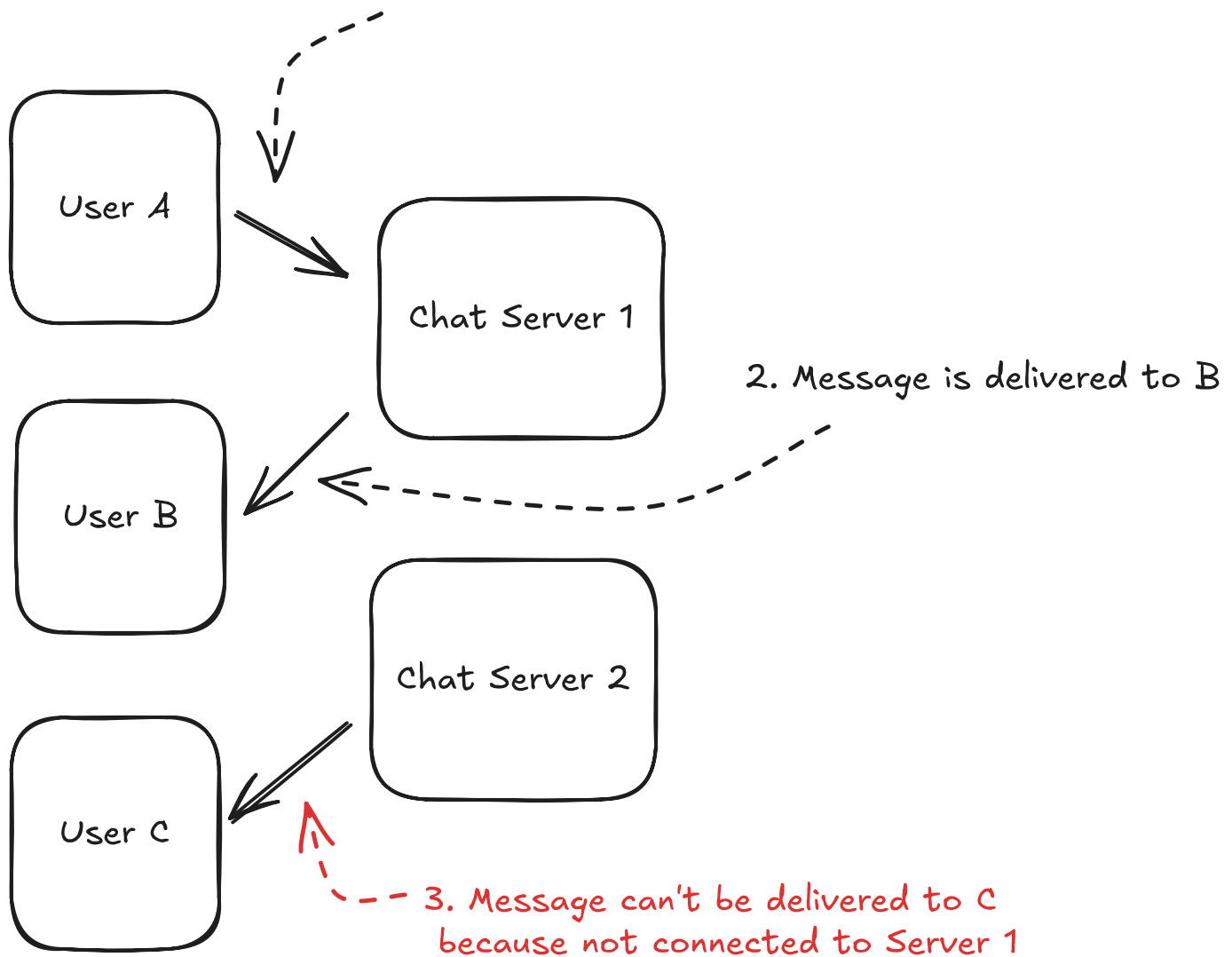
If we have 1b users, we might expect 200m of them to be connected at any one time. Whatsapp famously served 1-2m users per host, but this will require us to have hundreds of chat servers. That's a lot of simultaneous connections (!).



Note that I've included some back-of-the-envelope calculations here. Your interviewer will likely expect them, but you'll get more mileage from your calculations by doing them just-in-time: when you need to figure out a scaling bottleneck.

Adding more chat servers also introduces some new problems: now the sending and receiving users might be connected to different hosts. If User A is trying to send a message to User B and C via Chat Server 1, but User C is connected to Chat Server 2, we're going to have a problem.

1. User A tries to send a message to B and C



Host Confusion

The issue is one of routing: we need to route messages to the right Chat Servers in order to deliver them. We have a few options here which are discussed in greatest depth in the [Realtime Updates Deep Dive](#).

Bad Solution: Naively horizontally scale



Bad Solution: Keep a kafka topic per user



Good Solution: Consistent Hashing of Chat Servers



Great Solution: Offload to Pub/Sub



Should We Partition By Chat Or By User?

You may have the idea: "why do we have the pub/sub topics/channels be per user rather than per chat?", or maybe your interviewer asks you about this! The right choice is going to depend on (a) the number of chats per user, and (b) the size of those chats. Let's consider two scenarios to make this clear:

Scenario 1: Users have 250 chats each, but each chat has 1 other participant (1:1 chats).



Scenario 2: Users have 1 chat each, but each chat has 100 participants.



So which is right for this problem? Whatsapp is dominated by 1:1 chats. Having hundreds of redundant channels stresses Redis for little benefit. We also explicitly put a limit on the number of participants per chat to 100.

For more senior candidates you might be asked to discuss additional efficiencies you can eke out here. This is an example of a "celebrity problem" where an uncommon edge case (large chats) is disproportionately impacting the system. If this is a problem you want to solve, a good solution is to adaptively change the partitioning strategy based on the size of the chat.

When users connect, we'll list out all the chats they are part of which are larger than some threshold (say, 25 users). They'll subscribe to channels for *those chats specifically* in addition to the user-level channels. When a message is sent, if the chat is larger than the threshold, we'll publish to the chat-level channel instead of the user-level channel. There's edge cases here:

you need to be careful that you give time for the chat servers to subscribe when the chat size changes, so you might be publishing to both channels for a short time.

2) What do we do to handle multiple clients for a given user?

To this point we've assumed a user has a single device, but many users have multiple devices: a phone, a tablet, a desktop or laptop - maybe even a work computer. Imagine my phone had received the latest message but my laptop was off. When I wake it up, I want to make sure that all of the latest messages are delivered to my laptop so that it's in sync. We can no longer rely on the user-level "Inbox" table to keep track of delivery!

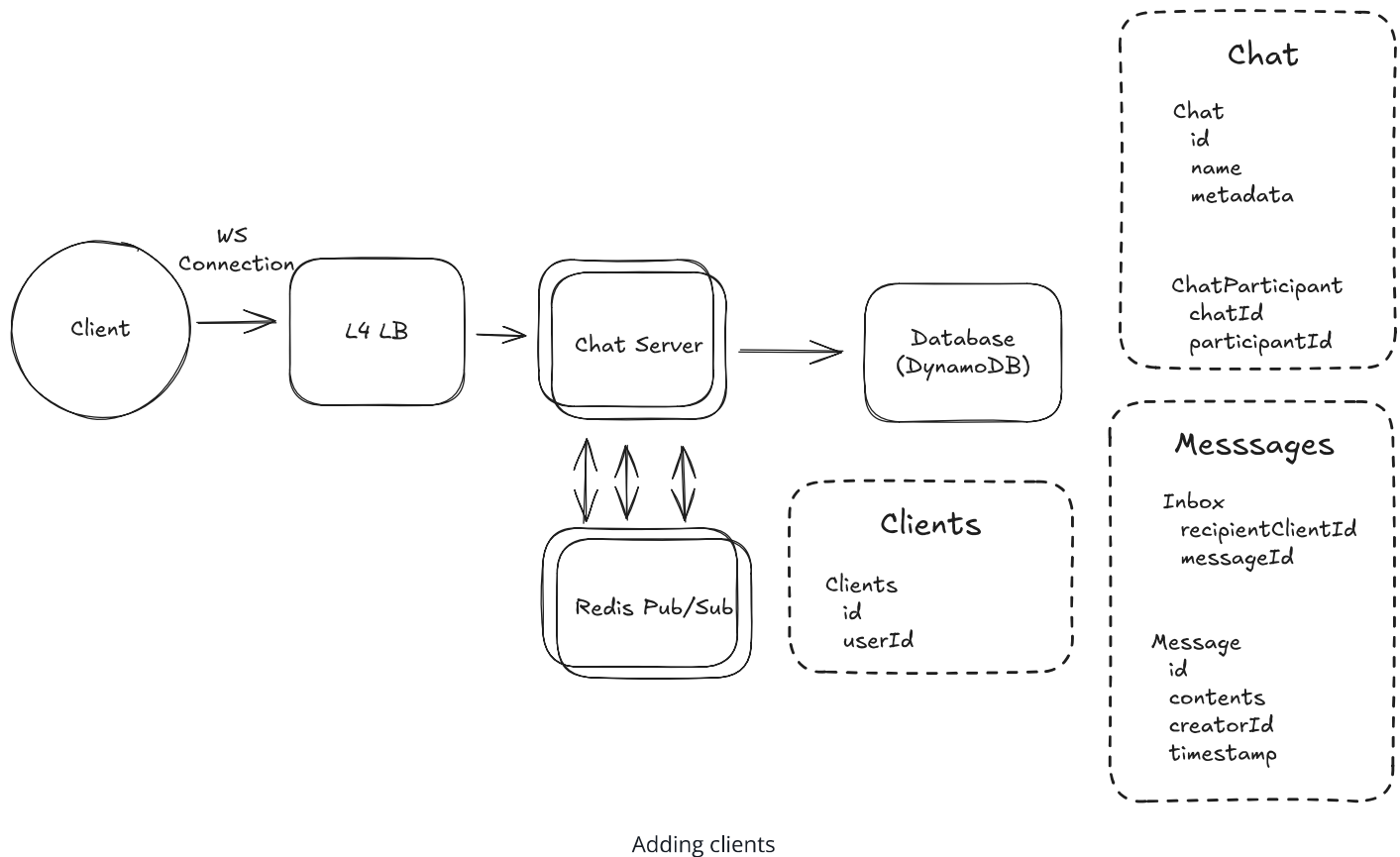
Having multiple clients/devices introduces some new problems:

- First, we'll need to add a way for our design to resolve a user to 1 or more clients that may be active at any one time.
- Second, we need a way to deactivate clients so that we're not unnecessarily storing messages for a client which does not exist any longer.
- Lastly, we need to update our message delivery system so that it can handle multiple clients.

Let's see if we can account for this with minimal changes to our design.

- We'll need to create a new `clients` table to keep track of clients by user id.
- When we look up participants for a chat, we'll need to look up all of the clients for that user.
- We'll need to update our `Inbox` table to be per-client rather than per-user.
- When we send a message, we'll need to send it to all of the clients for that user.
- On the pub/sub side, nothing needs to change. Chat servers will continue to subscribe to a topic with the `userId`.

We'll probably want to introduce some limits (3 clients per account) to avoid blowing up our storage and throughput.



3) What happens if the WebSocket connection fails?

Users often sit on poor network connections. The WebSocket may technically be open, but the connection is functionally severed—we won't know until we try to send a message and it times out. TCP keepalives can take minutes to detect a dead connection, which is far too slow for a chat app. How can we make sure our users aren't impacted?

Bad Solution: Rely on TCP Timeouts



Good Solution: ACK Timeouts with Server-Side Retry



Great Solution: Application-Level Heartbeats



4) What happens if Redis fails to deliver a message?

Redis Pub/Sub is "at most once"—if there's no subscriber listening or Redis has a transient failure, the message is lost. We've already handled durability by writing to the Inbox before

publishing to Pub/Sub (importantly **this means all messages will eventually get delivered**), but how do we ensure connected clients quickly receive messages that Pub/Sub dropped?

Good Solution: Periodic Polling



Good Solution: Sequence Numbers per Chat with Gap Detection



Great Solution: Piggyback Sequence on Heartbeats



In practice, most production systems combine these strategies: heartbeats detect dead WebSocket connections, sequence numbers on to detect missed messages, and periodic polling serves as a final backstop.

5) How do we handle out-of-order messages?

The simple answer is: **we don't**, or at least not directly.

Out-of-order messages are a fact of life in distributed systems and engineering such that messages are processed in the exact order they were sent is actually a considerable amount of additional complexity. We'd need to have delays to ensure we have time for late messages to arrive and a re-ordering mechanism to handle them. You can see an example in our [Flink](#) deep dive where Flink's Bounded Out-Of-Orderness Watermark Strategy effectively waits for late messages to arrive before processing them.

But for apps like this, it's not how they work! Users would rather see new messages as quickly as possible than guarantee order. So what do we do?

All of the Chat Servers will sync their time over NTP (Network Time Protocol). This doesn't guarantee perfect time, but it's pretty good. When a message arrives on the Chat Server, we'll "stamp" it with the time it was received. Then, when clients retrieve messages, they'll have the timestamp that they were received by the server. When we display messages, we display them ordered by this time. Messages have a consistent ordering across all clients even if they arrive in a different order than they are displayed.

On occasion, this will mean a message will pop-in "above" another message that was actually sent later. Users find this acceptable!

6) How can we handle a "last seen" functionality?

Our interviewer asks "how can we add a 'last seen' functionality to chats, which shows you when the other person was last online?"

Ideally, we want a solution that is both efficient and scalable.

Bad Solution: Write to DB on every heartbeat



Great Solution: Utilize Active Connections



What is Expected at Each Level?

Ok, that was a lot. You may be thinking, "how much of that is actually required from me in an interview?" Let's break it down.

Mid-level

Breadth vs. Depth: A mid-level candidate will be mostly focused on breadth (80% vs 20%). You should be able to craft a high-level design that meets the functional requirements you've defined, but many of the components will be abstractions with which you only have surface-level familiarity.

Probing the Basics: Your interviewer will spend some time probing the basics to confirm that you know what each component in your system does. For example, if you use websockets, expect that they may ask you what it does and how they work (at a high level). In short, the interviewer is not taking anything for granted with respect to your knowledge.

Mixture of Driving and Taking the Backseat: You should drive the early stages of the interview in particular, but the interviewer doesn't expect that you are able to proactively recognize problems in your design with high precision. Because of this, it's reasonable that they will take over and drive the later stages of the interview while probing your design.

The Bar for Whatsapp: For this question, an E4 candidate will have clearly defined the API, landed on a high-level design that is functional and meets the requirements. Their scaling solution will have rough edges but they'll have some knowledge of its flaws.

Senior

Depth of Expertise: As a senior candidate, expectations shift towards more in-depth knowledge — about 60% breadth and 40% depth. This means you should be able to go into technical details in areas where you have hands-on experience. It's crucial that you demonstrate a deep understanding of key concepts and technologies relevant to the task at hand.

Advanced System Design: You should be familiar with advanced system design principles. For example, knowing about the consistent hashing for this problem is essential. You're also expected to understand the mechanics of long-running sockets. Your ability to navigate these advanced topics with confidence and clarity is key.

Articulating Architectural Decisions: You should be able to clearly articulate the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability. You justify your decisions and explain the trade-offs involved in your design choices.

Problem-Solving and Proactivity: You should demonstrate strong problem-solving skills and a proactive approach. This includes anticipating potential challenges in your designs and suggesting improvements. You need to be adept at identifying and addressing bottlenecks, optimizing performance, and ensuring system reliability.

The Bar for Whatsapp: For this question, E5 candidates are expected to speed through the initial high level design so you can spend time discussing, in detail, scaling and robustness issues in the design. You should also be able to discuss the pros and cons of different architectural choices (like partitioning by chat or user), especially how they impact scalability, performance, and maintainability.

Staff+

Emphasis on Depth: As a staff+ candidate, the expectation is a deep dive into the nuances of system design — I'm looking for about 40% breadth and 60% depth in your understanding.

This level is all about demonstrating that, while you may not have solved this particular problem before, you have solved enough problems in the real world to be able to confidently design a solution backed by your experience.

You should know which technologies to use, not just in theory but in practice, and be able to draw from your past experiences to explain how they'd be applied to solve specific problems effectively. The interviewer knows you know the small stuff so you can breeze through that at a high level so you have time to get into what is interesting.

High Degree of Proactivity: At this level, an exceptional degree of proactivity is expected. You should be able to identify and solve issues independently, demonstrating a strong ability to recognize and address the core challenges in system design. This involves not just responding to problems as they arise but anticipating them and implementing preemptive solutions. Your interviewer should intervene only to focus, not to steer.

Practical Application of Technology: You should be well-versed in the practical application of various technologies. Your experience should guide the conversation, showing a clear understanding of how different tools and systems can be configured in real-world scenarios to meet specific requirements.

Complex Problem-Solving and Decision-Making: Your problem-solving skills should be top-notch. This means not only being able to tackle complex technical challenges but also making informed decisions that consider various factors such as scalability, performance, reliability, and maintenance.

Advanced System Design and Scalability: Your approach to system design should be advanced, focusing on scalability and reliability, especially under high load conditions. This includes a thorough understanding of distributed systems, load balancing, caching strategies, and other advanced concepts necessary for building robust, scalable systems.

The Bar for Whatsapp: For a staff+ candidate, expectations are high regarding depth and quality of solutions, particularly for the complex scenarios discussed earlier. Great candidates are going 2 or 3 levels deep to discuss failure modes, bottlenecks, and other issues with their design. There's ample discussion to be had around fault tolerance, database optimization, regionalization and cell-based architecture and more.

References

- What Happens When You Make a Move in Lichess

Test Your Knowledge

Answer the question below to find your gaps.

Question 1 of 15

Which is NOT a benefit of using acknowledgment patterns in message systems?

- 1 Enables retry logic
- 2 Prevents message loss during network failures
- 3 Reduces server memory usage
- 4 Confirms message delivery to the recipient